Cornerstone Project

LAB 4

Bytecode Virtual Machine

Technical Report

Submitted by: Monit Vaghela (2025MCS2117)
Anunay Sharma (2025MCS2095)

Submitted to: Prof. Kolin Paul

Date: January 7, 2026

Department of Computer Science and Engineering
Indian Institute of Technology Delhi

1 Problem Description

Using the C programming language, this assignment entails designing and implementing a stack-based bytecode virtual machine along with a corresponding assembler. Programs written in a custom assembly language are converted by the assembler into bytecode, which the virtual machine interprets and runs.

Key Objectives:

- To translate assembly instructions into executable bytecode, implement an assembler.
- Create a virtual machine that is stack-based in order to decipher and run bytecode programs.
- Provide support for control flow, memory access, arithmetic operations, and program termination instructions.
- Using benchmarking support, gather execution metrics like instruction count and execution time distribution.
- Provide clear error messages and ensure robust error handling.

2 Architecture

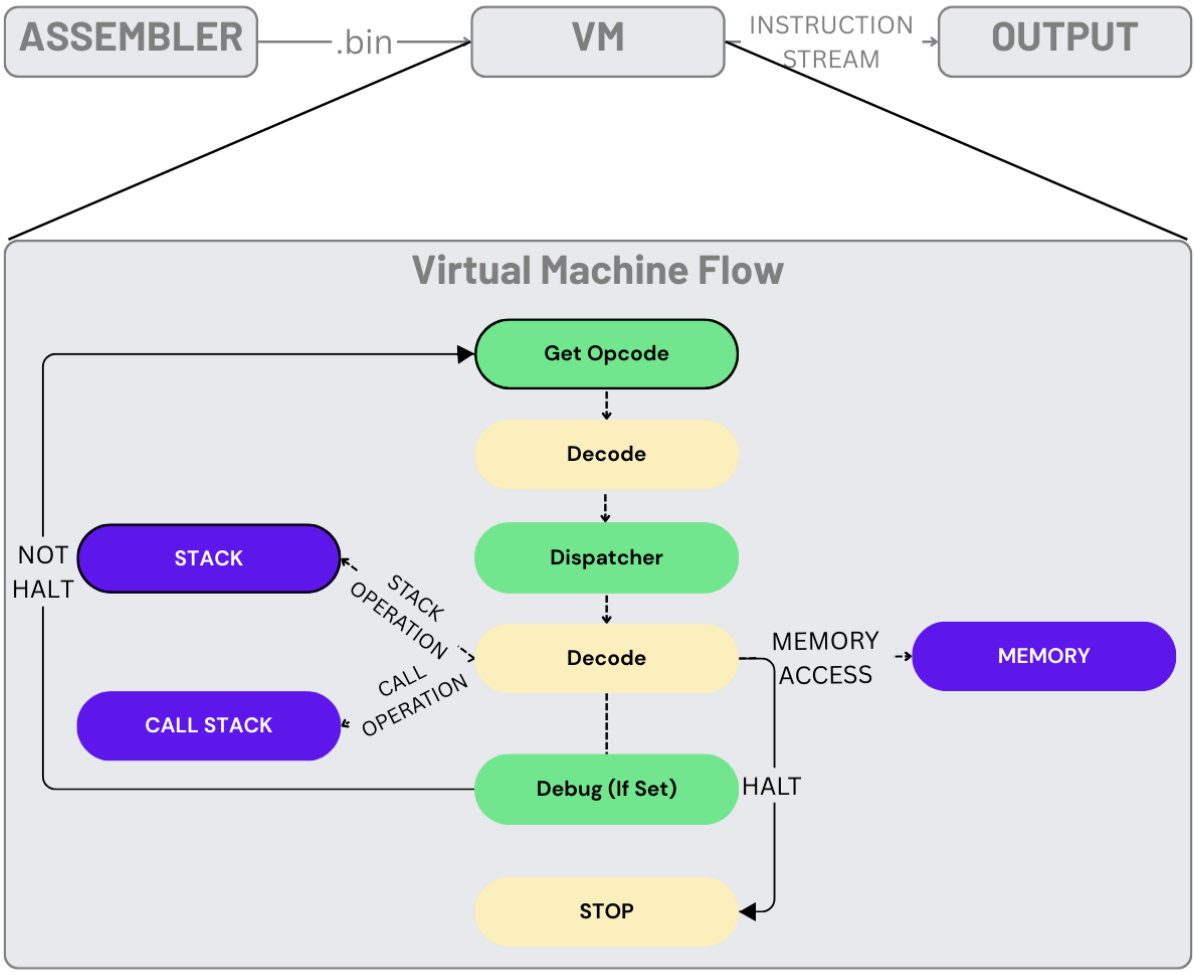

Figure 1: Virtual Machine Architecture and Execution Flow

3 Dispatch Strategy

This Virtual Machine uses a Switch based dispatch stratergy

- In this approach, the fetched opcode is decoded and dispatched to the corresponding instruction handler using a switch statement.
- Each case in the switch represents one instruction and contains the logic required to execute that instruction.

```
opcode = vm->code[vm->pc++];

switch (opcode) {
    case ADD:
        /* pop two values, add them, push result */
        break;

    case LOAD:
        /* load value from memory and push to stack */
        break;

    case HALT:
        vm->running = 0;
        break;
}
```

In this example, the opcode is fetched from the bytecode, and the `switch` statement dispatches execution to the correct handler based on the instruction type.

4 Opcodes

Opcode	Syntax	Size (bytes)	Description
Stack Operations			
PUSH	PUSH <value>	5	Push immediate value onto the stack
POP	POP	1	Remove top value from the stack
DUP	DUP	1	Duplicate top stack value
Arithmetic Operations			
ADD	ADD	1	Add top two stack values
SUB	SUB	1	Subtract top two stack values
MUL	MUL	1	Multiply top two stack values
DIV	DIV	1	Divide top two stack values
CMP	CMP	1	Compare top two stack values
Memory Operations			
LOAD	LOAD <addr>	5	Load value from memory address
STORE	STORE <addr>	5	Store value to memory address
Control Flow Operations			
JMP	JMP <addr>	5	Unconditional jump
JZ	JZ <addr>	5	Jump if zero
JNZ	JNZ <addr>	5	Jump if not zero
CALL	CALL <addr>	5	Call function
RET	RET	1	Return from function
HALT	HALT	1	Terminate program execution

Table 1: Virtual Machine Instruction Set

5 Design of call frames and return mechanism.

The Virtual machine reuses the stack structure to create a new CAllSTACK whose sole purpose is to store the return address how storing of return address works

CALL

- Firstly CALL Opcode is fed to the dispatched
- Dispatcher immediately checks whether the given address is out of bounds or not
- If not then it stores the current PC value on top of CALLSTACK and updates the PC to the address.

RET

- Check whether CALL was executed before RET
- If so then it POP the top of stack and updates to that address

6 Features

- Stack-based execution model
- Custom instruction set (as per Lab-4 specification)
- Separate stack and call stack
- Memory load/store support
- Structured error message → "[VM Error / PC] Error message"
- Assembler (.asm → .bin)
- Multiple execution modes
- Debug mode
- Wide range of test cases.(array addition using loop, GCD)

7 Limitations and possible Future Enhancements

7.1 Limitations

- A switch-based instruction dispatch strategy may introduce performance overhead for large programs.
- Execution timing is based on CPU time and is not cycle-accurate.
- The instruction set is limited and does not support advanced features such as dynamic memory allocation or exception handling.
- For the most part Bytecode is assumed to be valid
- .sh script only runs -ae mode without debug.

7.2 Limitations

- Implement faster instruction dispatch strategies (e.g., function pointer tables or computed goto).
- Add cycle-accurate performance measurement using hardware counters.
- Extend the instruction set to support more complex operations.

8 Version Control Activity GitHub

Git was used to track development progress and collaboration throughout the project.

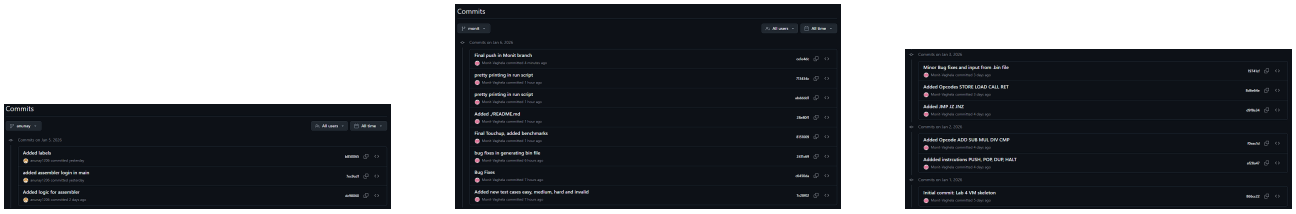

Figure 2: Git activity showing version control usage during development

9 Test Case Validation

The following screenshots demonstrate successful execution of representative test cases.

```
monit@ubuntu-vm:~/COD7001-Assignment4$ ./VM ./tests/easy/test6.asm -ae
FINAL VM STATE

==================================================
                VIRTUAL MACHINE STATE
==================================================
PC      : 6
OPCODE  : 0xFF
RUNNING : NO

---------------------- STACK -----------------------
[99, ]

------------------ CALL STACK ----------------------
[]

---------------------- MEMORY ----------------------
[ADDR   2] = 99
==================================================
```

```
monit@ubuntu-vm:~/COD7001-Assignment4$ ./VM ./tests/medium/test2.asm -ae

==================================================
                VIRTUAL MACHINE STATE
==================================================
PC      : 33
OPCODE  : 0xFF
RUNNING : NO

---------------------- STACK -----------------------
[2004189184, ]

------------------ CALL STACK ----------------------
[]

---------------------- MEMORY ----------------------
[ADDR   0] = 2004189184
[ADDR   1] = 1
==================================================
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

FINAL VM STATE

==================================================
                VIRTUAL MACHINE STATE
==================================================
PC      : 75
OPCODE  : 0xFF
RUNNING : NO

---------------------- STACK -----------------------
[]

------------------ CALL STACK ----------------------
[]

---------------------- MEMORY ----------------------
[ADDR   0] = 1
[ADDR   1] = 2
[ADDR   2] = 3
[ADDR   3] = 4
[ADDR   4] = 5
[ADDR  50] = 6
[ADDR  51] = 7
[ADDR  52] = 8
[ADDR  53] = 9
[ADDR  54] = 10
[ADDR 100] = 7
[ADDR 101] = 9
[ADDR 102] = 11
[ADDR 103] = 13
[ADDR 104] = 15
==================================================
monit@ubuntu-vm:~/COD7001-Assignment4$
```

```
monit@ubuntu-vm:~/COD7001-Assignment4$ ./VM ./tests/invalid/test6.asm -ae
Initializing virtual machine
[VM Error / 0]: Out of Bound Address


FINAL VM STATE

==================================================
                VIRTUAL MACHINE STATE
==================================================
PC      : 0
OPCODE  : 0xFF
RUNNING : NO

---------------------- STACK -----------------------
[]

------------------ CALL STACK ----------------------
[]

---------------------- MEMORY ----------------------
NO Initialized Memory
==================================================
```

Figure 3: Representative test case outputs